Laboratorio: Bases de Datos Embebidas

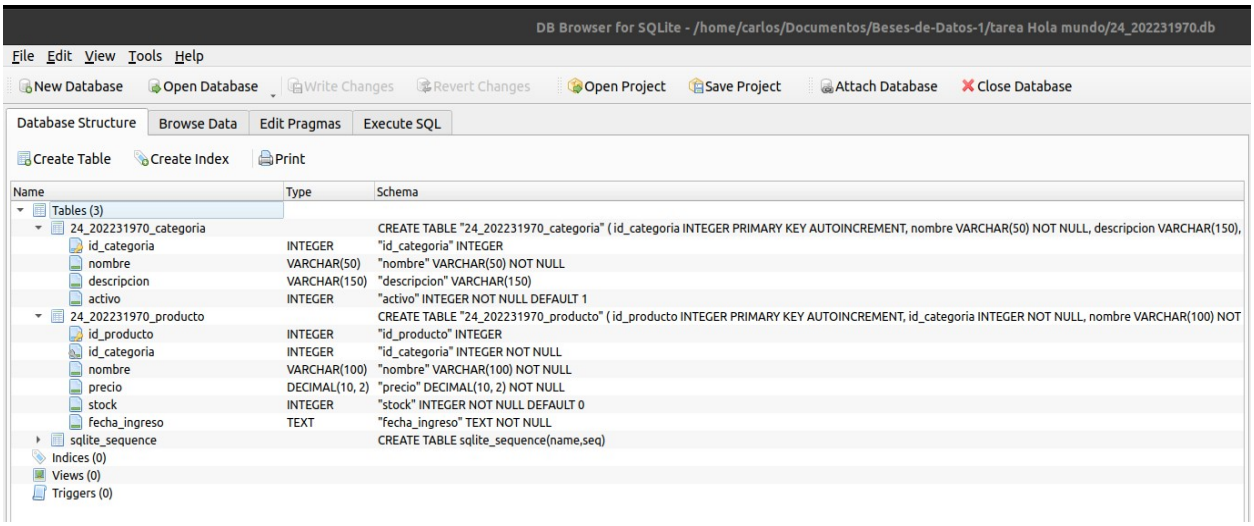
Hola Mundo con SQLite

ID: #24 | Carnae: 202231970

Base de datos: 24_202231970.db

Motor: SQLite 3 | 2025

1.Archivo .db



2.Tablas

Table: 24_202231970_categoria

	id_categoria	nombre	descripcion	activo
	Filter	Filter	Filter	Filter
1	1	Electrónica	Dispositivos y accesorios electrónicos	1
2	2	Hogar	Artículos para el hogar y cocina	1
3	3	Deportes	Equipos y ropa deportiva	1

Table: 24_202231970_producto

	id_producto	id_categoria	nombre	precio	stock	fecha_ingreso
	Filter	Filter	Filter	Filter	Filter	Filter
1	1	1	Audífonos Bluetooth	314.99	15	2025-01-10
2	2	1	Teclado Mecánico	539.1	8	2025-02-05
3	3	1	Mouse Inalámbrico	170.55	20	2025-02-20
4	4	2	Licuadora 600W	250	12	2025-01-15
5	5	2	Set de Ollas	480	5	2025-03-01
6	6	3	Pelota de Fútbol	120	30	2025-01-28

3.Select inicial

SQL 1

```sql
SELECT p.id_producto, c.nombre AS categoria,
       p.nombre AS producto, p.precio, p.stock
FROM "24_202231970_producto" p
JOIN "24_202231970_categoria" c ON p.id_categoria = c.id_categoria;
```

	id_producto	categoria	producto	precio	stock
1	1	Electrónica	Audífonos Bluetooth	314.99	15
2	2	Electrónica	Teclado Mecánico	539.1	8
3	3	Electrónica	Mouse Inalámbrico	170.55	20
4	4	Hogar	Licuadora 600W	250	12
5	5	Hogar	Set de Ollas	480	5
6	6	Deportes	Pelota de Fútbol	120	30

4.Select despues de UPDATE — 10% descuento a productos de Electrónica

SQL 1

```sql
SELECT p.id_producto, c.nombre AS categoria,
       p.nombre AS producto,
       p.precio AS precio_con_descuento, p.stock
FROM "24_202231970_producto" p
JOIN "24_202231970_categoria" c ON p.id_categoria = c.id_categoria;
```

	id_producto	categoria	producto	precio_con_descuento	stock
1	1	Electrónica	Audífonos Bluetooth	283.49	15
2	2	Electrónica	Teclado Mecánico	485.19	8
3	3	Electrónica	Mouse Inalámbrico	153.5	20
4	4	Hogar	Licuadora 600W	250	12
5	5	Hogar	Set de Ollas	480	5
6	6	Deportes	Pelota de Fútbol	120	30

5. Script SQL Completo

Archivo de base de datos: 24_202231970.db

Tablas creadas con prefijo obligatorio del carnet:

Abrir +

24_202231970_script.sql
~/Documentos/Beses-de-Datos-1/tarea Hola mundo

```sql
-- PASO 1: CREAR TABLAS
CREATE TABLE IF NOT EXISTS "24_202231970_categoria" (
    id_categoria  INTEGER       PRIMARY KEY AUTOINCREMENT,
    nombre        VARCHAR(50)   NOT NULL,
    descripcion   VARCHAR(150),
    activo        INTEGER       NOT NULL DEFAULT 1
);

CREATE TABLE IF NOT EXISTS "24_202231970_producto" (
    id_producto   INTEGER        PRIMARY KEY AUTOINCREMENT,
    id_categoria  INTEGER        NOT NULL,
    nombre        VARCHAR(100)   NOT NULL,
    precio        DECIMAL(10,2)  NOT NULL,
    stock         INTEGER        NOT NULL DEFAULT 0,
    fecha_ingreso TEXT           NOT NULL,
    FOREIGN KEY (id_categoria) REFERENCES "24_202231970_categoria"(id_categoria)
);

-- PASO 2: INSERTAR REGISTROS
INSERT INTO "24_202231970_categoria" (nombre, descripcion, activo) VALUES
    ('Electrónica', 'Dispositivos y accesorios electrónicos', 1),
    ('Hogar',       'Artículos para el hogar y cocina',       1),
    ('Deportes',    'Equipos y ropa deportiva',               1);

INSERT INTO "24_202231970_producto" (id_categoria, nombre, precio, stock, fecha_ingreso) VALUES
    (1, 'Audífonos Bluetooth', 349.99, 15, '2025-01-10'),
    (1, 'Teclado Mecánico',    599.00,  8, '2025-02-05'),
    (1, 'Mouse Inalámbrico',   189.50, 20, '2025-02-20'),
    (2, 'Licuadora 600W',      250.00, 12, '2025-01-15'),
    (2, 'Set de Ollas',        480.00,  5, '2025-03-01'),
    (3, 'Pelota de Fútbol',    120.00, 30, '2025-01-28');

-- PASO 3: SELECT INICIAL
SELECT p.id_producto, c.nombre AS categoria,
       p.nombre AS producto, p.precio, p.stock
FROM "24_202231970_producto" p
JOIN "24_202231970_categoria" c ON p.id_categoria = c.id_categoria;

-- PASO 4: UPDATE — 10% descuento a productos de Electrónica
UPDATE "24_202231970_producto"
SET precio = ROUND(precio * 0.90, 2)
WHERE id_categoria = (
    SELECT id_categoria FROM "24_202231970_categoria"
    WHERE nombre = 'Electrónica'
);

-- PASO 5: SELECT DE VERIFICACIÓN (después del UPDATE)
SELECT p.id_producto, c.nombre AS categoria,
       p.nombre AS producto,
       p.precio AS precio_con_descuento, p.stock
FROM "24_202231970_producto" p
JOIN "24_202231970_categoria" c ON p.id_categoria = c.id_categoria;
```

6.Respuestas a las Preguntas

P1. ¿Qué es una base de datos embebida y en qué se diferencia de una cliente-servidor?

Una base de datos embebida es un motor que corre dentro del mismo proceso de la aplicación, sin necesitar un servidor separado, instalación de servicios ni puertos de red. SQLite es el ejemplo más conocido: toda la base de datos es un único archivo .db.

Característica	SQLite (Embebida)	PostgreSQL / MySQL (C-S)
Proceso	Dentro de la app	Servidor separado
Red	No requiere red	Requiere TCP/IP
Concurrencia	1 escritor a la vez	Múltiples clientes
Instalación	Solo el archivo .db	Instalación + config del servidor
Uso típico	Apps móviles, prototipos	Sistemas web, empresariales

P2. ¿Qué archivo o carpeta genera SQLite al crear una base de datos?

SQLite genera un único archivo binario: 24_202231970.db

Ese archivo contiene todo en su interior:

- Encabezado de 100 bytes con versión, tamaño de página y encoding.
- Páginas de datos en estructura B-Tree.
- Catálogo del sistema (sqlite_master) con las definiciones de tablas e índices.
- Diario de transacciones (archivo -journal temporal, solo mientras hay una transacción activa).

P3. ¿Qué esquema se crea por defecto y cuál es su propósito?

SQLite no tiene esquemas nombrados como H2 o Derby. En su lugar usa sqlite_master (también llamado sqlite_schema en versiones recientes) como catálogo del sistema.

Para verlo se ejecuta:

```
SELECT type, name FROM sqlite_master WHERE type IN ('table','index');
```

Resultado obtenido en este ejercicio:

type	name
table	24_202231970_categoria
table	24_202231970_producto
table	sqlite_sequence

sqlite_sequence es una tabla interna que SQLite crea automáticamente al usar AUTOINCREMENT. Lleva el registro del último ID asignado en cada tabla.

P4. ¿Qué tipos de datos soporta SQLite?

SQLite usa un sistema de tipos dinámicos con 5 clases de almacenamiento (type affinity):

Tipo / Afinidad	Descripción	Ejemplo de uso
INTEGER	Entero con signo (1–8 bytes)	id_producto INTEGER PRIMARY KEY AUTOINCREMENT
REAL	Flotante IEEE 754 (8 bytes)	precio REAL → 349.99
TEXT	Cadena UTF-8/UTF-16	nombre TEXT NOT NULL → 'Audífonos'
BLOB	Datos binarios sin modificar	foto BLOB → imagen del producto en bytes
NUMERIC	INTEGER o REAL según el valor	DECIMAL(10,2) → se almacena como NUMERIC

P5. ¿Qué ocurre si ejecutas un UPDATE sin WHERE?

Se actualizan ABSOLUTAMENTE TODOS los registros de la tabla, sin confirmación y sin posibilidad de deshacer (si no hay transacción activa).

Demostración ejecutada en el ejercicio:

```
UPDATE "24_202231970_producto" SET stock = stock + 100;  -- SIN WHERE
```

id_producto	nombre	stock ANTES	stock DESPUÉS
1	Audífonos Bluetooth	15	115
2	Teclado Mecánico	8	108
3	Mouse Inalámbrico	20	120
4	Licuadora 600W	12	112
5	Set de Ollas	5	105
6	Pelota de Fútbol	30	130

P6. ¿Qué es una restricción NOT NULL y cuándo usarla?

NOT NULL es una restricción de integridad que prohíbe que una columna almacene el valor NULL. Si se intenta insertar/actualizar con NULL en esa columna, SQLite lanza un error y rechaza la operación.

Ejemplo en este ejercicio:

```
nombre  VARCHAR(100)  NOT NULL,   -- producto siempre debe tener nombre
precio  DECIMAL(10,2) NOT NULL    -- no puede existir un precio vacío
```

¿Cuándo usarla?

- En columnas imprescindibles: nombre, precio, fecha de creación.
- En claves foráneas (id_categoria NOT NULL) para garantizar integridad referencial.
- En campos de auditoría: usuario, fecha de modificación.

¿Cuándo NO usarla?

- En datos genuinamente opcionales: descripción larga, teléfono secundario, foto de perfil.

P7. ¿En qué escenario elegirías SQLite sobre H2 y Derby?

Elegiría SQLite en los siguientes escenarios:

Aplicaciones móviles y de escritorio

SQLite viene integrado en Android e iOS. Una app de inventario o diario personal no necesita servidor: el archivo .db viaja con la app.

Prototipado rápido

No hay que instalar nada ni configurar puertos. Se crea el archivo, se ejecuta el SQL y listo. Para un MVP o proyecto de laboratorio es la opción más simple y directa.

Portabilidad total

El archivo 24_202231970.db puede copiarse a cualquier OS (Windows, Linux, macOS, Android) sin conversión. Ni H2 ni Derby ofrecen esa portabilidad de un solo archivo.

Scripts de análisis de datos

Python (pandas), R y otras herramientas pueden leer y escribir directamente un .db sin servidor. Ideal para pipelines de datos ligeros.

¿Cuándo NO elegir SQLite?

Si el proyecto requiere múltiples usuarios escribiendo simultáneamente, procedimientos almacenados complejos, o integración profunda con Java EE, entonces H2 o Derby serían mejores opciones por su mayor compatibilidad con el estándar SQL y mejor manejo de concurrencia.